

PumpScript

D5 – Retrospective & Self-Assessment (Part 2)

CSE 341 – Concepts of Programming Languages – Gebze Technical University – 22 May 2026
Yusuf Eren Nalbant (220104004049)

Retrospective

If I were starting over, I would decide the final language scope earlier and keep D1, D2, and D3 synchronized from the first day. The hardest part was not writing syntax rules; it was choosing a small DSL design that is useful, implementable, and defensible with Sebesta terminology. The fitness domain helped because constructs such as **exercise**, **routine**, and **rest N seconds** are naturally domain-specific, but the first versions still tried to include too many generic programming features.

The most important scoped-out features are arrays, assignment statements, mutable local variables, and boolean connectives such as **and/or**. I intentionally kept them out because they would make the type checker, environment model, and semantics longer than necessary. The final language is smaller but easier to explain: values come from literals and parameters, routines are procedures with no return values, and exercise blocks are checked against a fixed schema. The single hardest design decision was the type-system boundary for exercise records. I chose a named exercise record form with exact field checking instead of general structural records, accepting less flexibility in exchange for safer domain meaning.

Self-Assessment Against the Rubric

Dimension	Grade	Justification
Correctness	A-	The final lexer, parser, type checker, and interpreter run all three valid examples and reject the required type error. I also fixed the examples so that if and repeat are actually executed.
Design Justification	A-	Major choices are justified with trade-offs: strong typing, only int → float coercion, name equivalence, no assignment, and no boolean connectives.
Use of Sebesta Framework	A-	D1 uses EBNF, operational semantics, strong typing, type equivalence, coercion, static scoping, and stack-dynamic lifetime terminology.
Originality	B+	The workout-routine DSL has genuine domain constructs, especially exercise and rest . It is still intentionally small and uses conventional control structures.
Code Quality	A-	Components are separated into lexer, parser, AST nodes, type checker, interpreter, and main entry point. The code is readable and line-numbered errors are useful.
Exam Performance	B+	I can trace the implementation and explain the main design decisions. The part I need to review most before the exam is writing new formal semantics rules by hand.

Overall reflection. I would grade my Part 2 submission as strong but not perfect. The final version is intentionally modest, but the language specification, implementation, and test report now describe the same language, which is the most important requirement for defending the project.